

React Fun with useState & Event Handling

Learn how React remembers things and responds when kids click, type, and interact!

1. Understanding State & Events

In regular static websites, content cannot change on its own—like a photo printed on paper! In modern React apps, we want live interactive components—like a digital game scoreboard or a clickable live dice!

What is "State"?

Think of State as your component's **Magic Piggy Bank** or live notebook! It remembers values that change over time as kids play with the app (e.g., game scores, favorite color, cheer counters).

What are "Events"?

Events are actions triggered by users—like clicking a button, typing in a box, or hovering a mouse! Events tell React: *"Hey, update the memory box now!"*

2. Step-by-Step Code Examples for Beginners

Example 1: Live IPL Fan Cheer Counter (+1 & Reset Buttons)

- Step 1:** Import the useState hook from React at the top.
- Step 2:** Declare a state variable cheers starting at 0.
- Step 3:** Use onClick event handlers to add +1 cheer or reset to 0!

```
import React, { useState } from 'react';

function CheerCounter() {
  // Syntax: const [stateValue, updaterFunction] = useState(initialValue);
  const [cheers, setCheers] = useState(0);

  return (
    <div style={{ textAlign: 'center', padding: '10px' }}>
      <h2>🏏 Live IPL Cheers: {cheers}</h2>
      <button onClick={() => setCheers(cheers + 1)}>+1 Cheer 🎉</button>
      <button onClick={() => setCheers(0)}>Reset 🔄</button>
    </div>
  );
}
```

Example 2: Interactive Light Switch (Toggle Boolean State)

- Step 1:** Set initial state isOn to false.
- Step 2:** Use boolean negation !isOn to flip between true and false on click!

```
import React, { useState } from 'react';

function LightSwitch() {
  const [isOn, setIsOn] = useState(false);

  return (
    <div>
      <h3>The Room is: {isOn ? '💡 BRIGHT & GLOWING!' : '🌙 DARK & SLEEPY'}</h3>
      <button onClick={() => setIsOn(!isOn)}>
        {isOn ? 'Turn Off 🔴' : 'Turn On 🟢'}
      </button>
    </div>
  );
}
```

Example 3: Magic Name Tag (Text Input with onChange Event)

1. **Step 1:** Track what kids type into an input box with name state.
2. **Step 2:** Listen to the onChange event and capture typed text using `e.target.value`.

```
import React, { useState } from 'react';

function MagicNameTag() {
  const [name, setName] = useState('');

  return (
    <div>
      <h3>Hello, {name === '' ? 'Little Coder' : name}! 🙌</h3>
      <input
        type="text"
        placeholder="Type your name..."
        onChange={(e) => setName(e.target.value)}
      />
    </div>
  );
}
```

Example 4: Favorite Superhero Selector (Updating Strings)

1. **Step 1:** Store a hero string in state (default: 'Spider-Man').
2. **Step 2:** Provide multiple choice buttons that update state to different hero names!

```
import React, { useState } from 'react';

function HeroSelector() {
  const [hero, setHero] = useState('Spider-Man 🕷️');

  return (
    <div>
      <h3>Current Hero: {hero}</h3>
      <button onClick={() => setHero('Spider-Man 🕷️')}>Spider-Man</button>
      <button onClick={() => setHero('Iron Man 💪')}>Iron Man</button>
      <button onClick={() => setHero('Wonder Woman 🦹')}>Wonder Woman</button>
    </div>
  );
}
```

3. Bug Buster: Avoid Common Oopsies! 🐛

⚠️ 3 Golden Rules to Prevent Bugs:

1. **Capital 'C' in onClick:** React uses camelCase! Writing `onClick` (lowercase) will fail quietly. Always write `onClick`.
2. **Use Function Parentheses:** Always call updater function e.g. `setCheers(cheers + 1)`. Never assign directly like `cheers = cheers + 1`!
3. **Always Import the Hook:** Always include `import { useState } from 'react';` at the very top line of your file!



STUDENT PRACTICE WORKSHEET & EXERCISES

Question 1: Multiple Choice

What happens in React if you write `onclick` with a lowercase 'c' instead of `onClick` ?

- A) It works perfectly fine.
- B) Nothing happens / React ignores the event because event names must be camelCase.
- C) The computer blows up into fireworks!

Question 2: Conceptual Understanding

In our lesson analogy, `useState` was compared to a "Magic Piggy Bank" or "Notebook". Explain why state is required in an interactive app.

Question 3: Code Analysis

Look at this code: `const [score, setScore] = useState(10);`

Identify what each of the three parts represents:

1. `score` : _____
2. `setScore` : _____
3. `10` : _____

Question 4: Step-by-Step Code Practice (Diwali Diya Mini-Project)

Fill in the missing step-by-step code snippets below to build a `DiwaliDiya` toggle component:

```
import React, { _____ } from 'react'; // Step 1: Fill hook name

function DiwaliDiya() {
  // Step 2: Create boolean state 'isGlowing' starting as false
  const [isGlowing, setIsGlowing] = _____(false);

  return (
    <div>
      <h1>{isGlowing ? "🔥 Diya is Glowing Bright!" : "🌑 Diya is Off"}</h1>
      <button _____={() => setIsGlowing(!isGlowing)}> // Step 3: Event name
        Toggle Diya
      </button>
    </div>
  );
}
```

Question 5: Step-by-Step Code Challenge (Pet Counter)

Write a complete React component named `PetCounter` with two buttons: `" +1 Dog 🐶 "` and `" -1 Dog 🐶 "`. Ensure the dog count never goes below 0!

Hint Use `setDogs(dogs > 0 ? dogs - 1 : 0)` for the minus button!

Question 6: Bug Hunting Challenge 🐛

Find 2 bugs in the following student code and rewrite it correctly:

```
<button onclick={setScore = score + 1}>Add Point</button>
```